



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

Subject: Programming with Java

Name: Husam Ali Abdalla Nour

Prn: 25070122086

ASSIGNMENT No: 1

TITLE: Develop an application based on Java environment setup, JDK/JRE/JVM, variables, methods and constructors. and demonstrate the implementation of the concepts through suitable examples.

Problem Statement

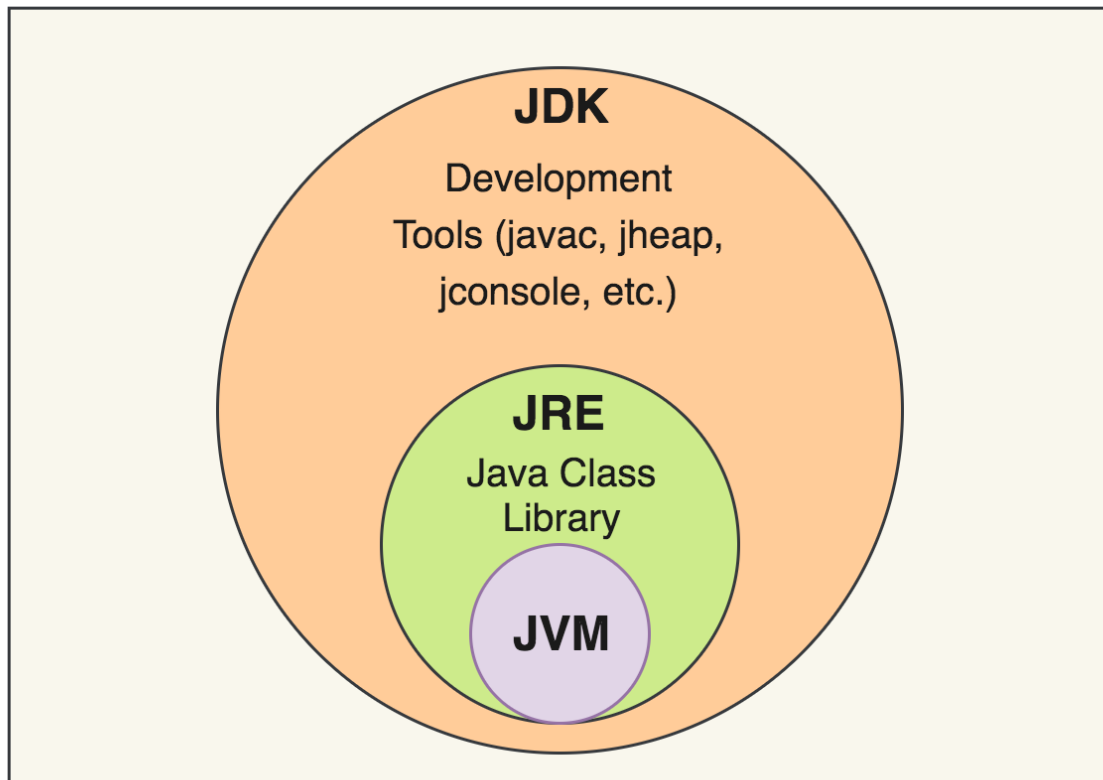
Develop an application based on Java environment setup, JDK/JRE/JVM, variables, methods and constructors. and demonstrate the implementation of the concepts through suitable examples.

Learning Objectives

Understand the theoretical concepts, implement them in Java programs, analyze outputs, and apply the concepts to solve practical programming problems.

Theory

Java environment setup, JDK/JRE/JVM, variables, methods and constructors. forms an important component of Java programming. Students should understand the underlying concepts, architecture, advantages, use cases, and implementation methodology. The exercise should focus on understanding the design principles, coding practices, execution flow, and practical applications of these concepts in software development. Detailed study should include syntax, APIs, execution process, and real-world applications.



This diagram illustrates the hierarchical relationship between **JDK**, **JRE**, and **JVM** using three nested circles.

The **JDK** is the largest component and contains all the tools needed for Java development, such as **java**, **jconsole**, and **jheap**.

Inside the JDK is the **JRE**, which provides the runtime environment for Java applications.

The JRE includes the **Java Class Library** and the **JVM**.

The **JVM** is responsible for executing Java bytecode and managing memory.

It also performs garbage collection and converts bytecode into machine code.

This structure shows that every JDK contains a JRE, and every JRE contains a JVM.

Therefore, developers use the JDK to build applications, while the JRE and JVM work together to run them successfully.

JDK

└─ JRE

└─ JVM

JDK (Outer Circle)

Contains:

- Development tools
- Compiler (`javac`)
- Debugger

- JConsole
- JHeap
- JAR tool
- JRE

Used by **Java developers**.

JRE (Middle Circle)

Contains:

- JVM
- Java Class Library

Used to **run Java applications**.

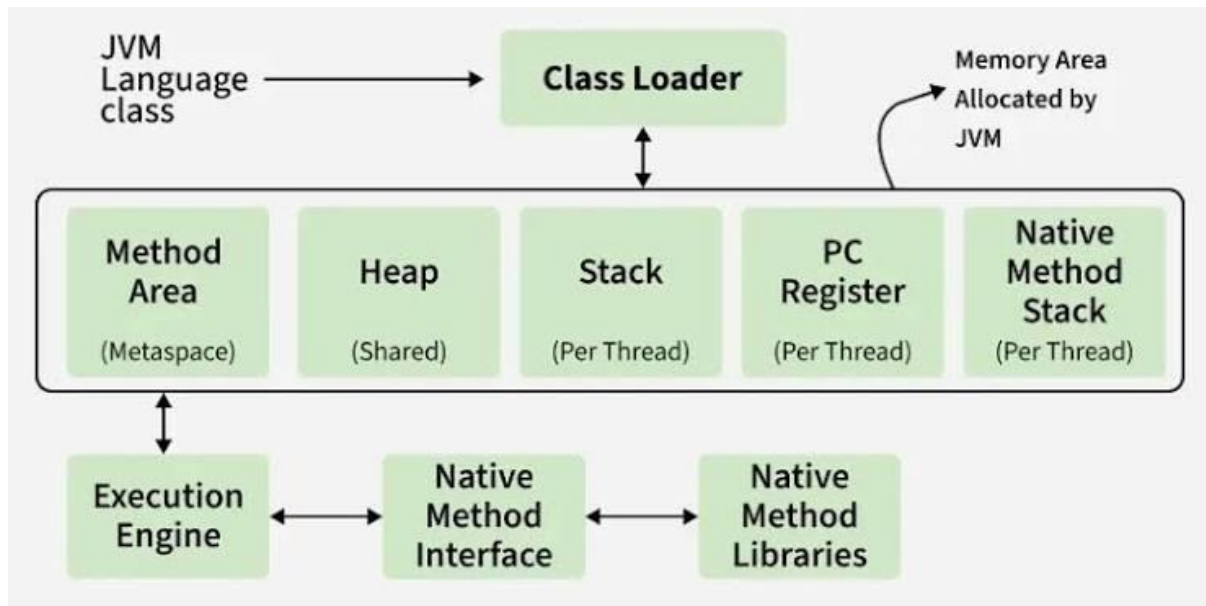
Users who only run Java programs need the JRE

JVM (Inner Circle)

The smallest component.

Responsibilities:

- Executes Java bytecode
- Allocates memory
- Performs Garbage Collection
- Provides platform independence



This diagram explains the internal architecture of the Java Virtual Machine (JVM).

The Class Loader loads Java class files into memory before execution.

The Method Area stores class information, methods, and static variables.

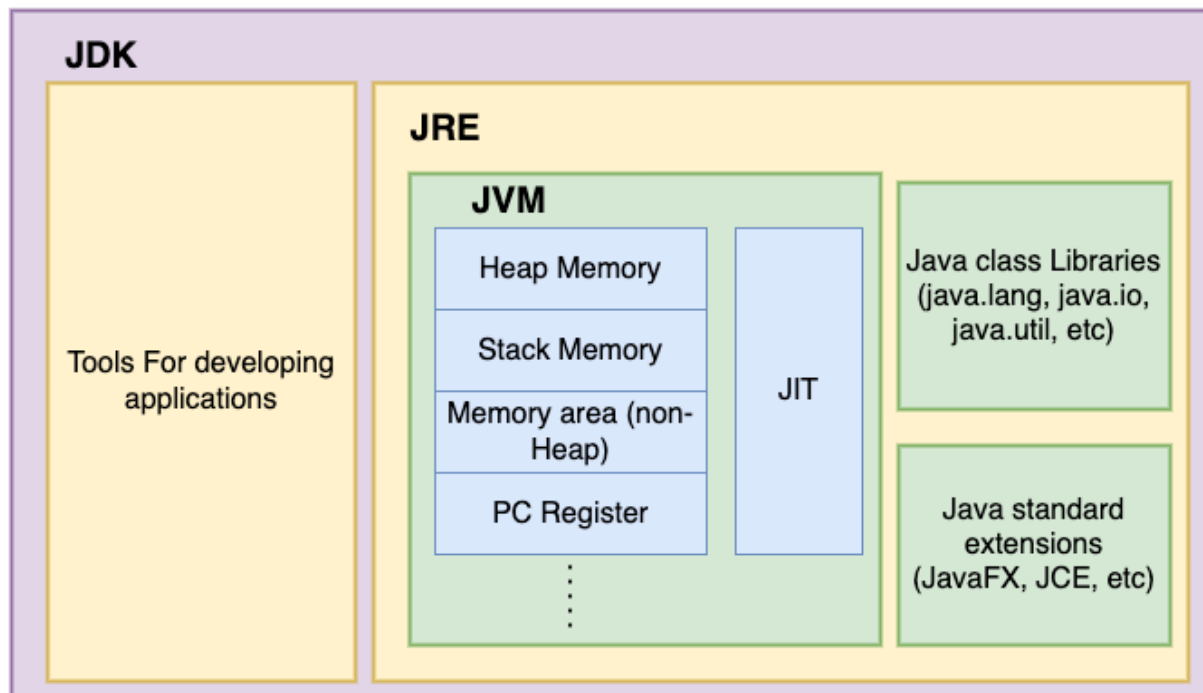
The Heap stores objects and is shared among all threads, while the Stack stores method calls and local variables for each thread.

The PC Register keeps track of the current instruction being executed.

The Native Method Stack supports methods written in languages such as C or C++.

The Execution Engine executes bytecode with the help of the JIT Compiler and Garbage Collector.

The Native Method Interface (JNI) allows Java programs to communicate with native libraries provided by the operating system.



This diagram shows the relationship between **JDK, JRE, and JVM**.

The **JDK (Java Development Kit)** is the complete package used by developers to write, compile, and debug Java programs.

Inside the JDK is the **JRE (Java Runtime Environment)**, which provides the environment required to run Java applications.

The JRE contains the **JVM (Java Virtual Machine)**, which executes Java bytecode.

The JVM manages memory using **Heap Memory, Stack Memory, Non-Heap Memory**, and the **PC Register**.

It also uses the **JIT (Just-In-Time) Compiler** to convert bytecode into machine code for faster execution.

The JRE includes Java class libraries such as **java.lang, java.util, and java.io**.

Together, these components allow Java programs to be developed, compiled, and executed efficiently on different operating systems.

- **JDK (Java Development Kit)** is the largest component.

- It contains **tools for developing Java applications** such as:
 - Java compiler
 - javadoc
 - jar
 - jdb
- Developers use the JDK to write, compile, and debug Java programs.

- **JRE (Java Runtime Environment)** is inside the JDK.
 - It provides everything needed to **run Java applications**.
 - It includes the JVM and Java libraries.
- **JVM (Java Virtual Machine)** is inside the JRE.
 - It executes Java bytecode.
 - It manages memory and converts bytecode into machine code using the **JIT (Just-In-Time) Compiler**.

Conclusion

In conclusion, the **JDK, JRE, and JVM** are the three main components of the Java platform, each serving a different purpose. The **JDK** provides the tools required to develop and compile Java programs, the **JRE** provides the environment to run them, and the **JVM** executes the bytecode while managing memory and system resources. Understanding their architecture and relationship helps developers learn how Java programs are compiled, executed, and made platform-independent. This knowledge is essential for writing efficient, reliable, and portable Java applications.

Questions

1. Explain the main concept used in Java environment setup, variables, and methods.

The Java environment setup involves installing the **JDK (Java Development Kit)**, setting the **JAVA_HOME** and **PATH** environment variables, and using an IDE such as Eclipse, IntelliJ IDEA, or VS Code to write and run Java programs. **Variables** are used to store data values such as integers, strings, and floating-point numbers. **Methods** are blocks of code that perform specific tasks, improve code organization, and allow code reuse. Together, they form the foundation of Java programming.

2. What are the advantages of using Java variables and methods?

- Variables make it easy to store and manage data.
- Methods reduce code duplication by allowing code reuse.
- They improve code readability and organization.
- They make programs easier to debug and maintain.
- They support modular programming, making large applications easier to develop.

3. How can this implementation be improved?

The implementation can be improved by using **meaningful variable and method names**, following Java naming conventions, adding **comments** for better understanding, handling **exceptions** to prevent errors, and dividing large programs into smaller reusable methods. Using object-oriented programming concepts such as classes and inheritance also improves code quality.

4. What are the real-world applications?

Java is widely used in many real-world applications, including:

- Android mobile application development.
- Web applications using Spring Boot and JSP.
- Banking and financial software.
- Enterprise business applications.
- E-commerce websites.
- Desktop applications.
- Cloud computing and distributed systems.
- Internet of Things (IoT) and embedded systems.